Applied Machine Learning Lab

B.Tech Semester 4

Experiment 06

Comparison of Classification Models on MNIST Dataset

Date: 21st Jan 2026

SAP ID: 590016154

Name: Vidisha

AIM

To implement and compare multiple classification models on the MNIST dataset and evaluate their performance using standard metrics.

THEORY

Image classification is a supervised learning task where models predict labels for images.
Classification Models: Logistic Regression, Decision Tree, Random Forest, SVM, KNN
Evaluation Metrics: Accuracy, Precision, Recall, F1-score
Confusion Matrix: Measures prediction performance
Feature Scaling: Important for SVM and Logistic Regression
High-dimensional Data: MNIST has 784 features

Commands/Code Used

```
# Import libraries

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier

from sklearn.svm import SVC

from sklearn.neighbors import KNeighborsClassifier


from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix



# Load dataset (using sklearn MNIST-like dataset)

from sklearn.datasets import fetch_openml


mnist = fetch_openml('mnist_784', version=1)

X = mnist.data

y = mnist.target.astype(int)
```

```
# Reduce size for faster execution
X = X[:10000]
y = y[:10000]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Models
models = {
   "Logistic Regression": LogisticRegression(max_iter=1000),
   "Decision Tree": DecisionTreeClassifier(),
   "Random Forest": RandomForestClassifier(),
   "SVM": SVC(),
   "KNN": KNeighborsClassifier()
}

results = {}

# Train and evaluate
for name, model in models.items():
   model.fit(X_train, y_train)
```

```
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='weighted')
    rec = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    results[name] = [acc, prec, rec, f1]

    print(f"\n{name}")
    print("Accuracy:", acc)
    print("Precision:", prec)
    print("Recall:", rec)
    print("F1 Score:", f1)

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(6,5))
    sns.heatmap(cm, cmap="Blues")
    plt.title(f"Confusion Matrix - {name}")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

# Model comparison
results_df = pd.DataFrame(results, index=["Accuracy", "Precision", "Recall", "F1 Score"]).T
```

```
results_df.plot(kind="bar", figsize=(10,6))

plt.title("Model Performance Comparison")

plt.xticks(rotation=45)

plt.ylabel("Score")

plt.show()
```

RESULTS

Confusion matrices generated for all models.
Performance comparison done using accuracy, precision, recall and F1-score.

CONCLUSION

Random Forest and SVM performed better. This experiment shows importance of model comparison and evaluation.